

Representación Computacional de Números Enteros

Diego Feroldi
feroldi@fceia.unr.edu.ar

Arquitectura del Computador*
Departamento de Ciencias de la Computación
FCEIA-UNR



* Actualizado 21 de julio de 2026 (D. Feroldi, feroldi@fceia.unr.edu.ar)

Índice

1. Introducción	1
2. Sistemas de numeración posicionales	1
3. Sistema binario	2
3.1. Definición del sistema binario	2
3.2. Representación binaria de enteros sin signo	2
3.3. Conversión entre sistemas	4
3.3.1. Binario a decimal	4
3.3.2. Decimal a binario	4
3.4. Rango de enteros sin signo	5
4. Otras representaciones	5
4.1. Sistema hexadecimal	6
4.2. Sistema octal	8
5. Unidades de información	9
5.1. Bit	9
5.2. Nibble	9
5.3. Byte	10
5.4. Palabra	11
5.5. Múltiplos del byte	11
6. Operaciones en sistema binario	12
6.1. Suma	12
6.2. Resta	13
6.3. Multiplicación	14
6.4. Bandera de acarreo (<i>Carry Flag</i>)	15
7. Operaciones en sistema hexadecimal	16
7.1. Suma	16
7.2. Resta	17
8. Representación de enteros con signo	18
8.1. Representación magnitud y signo	18
8.2. Complementos a la base y a la base menos uno	19
8.3. Representación en complemento a dos	20
8.4. Representación en complemento a uno	25

9. Operaciones en complemento a dos	29
9.1. Suma	29
9.2. Resta	31
9.3. Bandera de desbordamiento (<i>Overflow Flag</i>)	32
10. Operadores a nivel de bits	34
10.1. Operador AND	35
10.2. Operador OR	37
10.3. Operador XOR	38
10.4. Operador NOT	39
10.5. Propiedades del operador XOR	39
10.6. Desplazamiento de bits	40
A. Apéndices	44
A.1. Representación de caracteres	44
A.2. Representación de cadenas	45
A.3. Tamaños de tipos numéricos en arquitecturas x86-64	46

1. Introducción

La aritmética utilizada por las computadoras difiere de la que empleamos habitualmente, por lo que resulta fundamental comprender cómo se representan los datos en su interior antes de abordar temas más específicos de programación. La principal diferencia radica en que las computadoras solo pueden operar con números de precisión finita, generalmente fija. Además, la mayoría de ellas trabaja en sistema binario (base dos), a diferencia del sistema decimal (base diez) al que estamos acostumbrados. Por esta razón, en este apunte se abordará la representación de números enteros en las computadoras.

Observación

En este apunte nos enfocaremos exclusivamente en la representación de números enteros. La representación computacional de números reales, que requiere herramientas conceptuales adicionales, será abordada en el próximo apunte.

2. Sistemas de numeración posicionales

Para representar números, es habitual utilizar un sistema posicional de base 10, denominado *sistema decimal*. En este sistema se emplean diez símbolos, llamados dígitos: 0, 1, 2, 3, 4, 5, 6, 7, 8 y 9.

En un sistema posicional, el valor de cada dígito depende de su posición. Si un dígito a ocupa la posición n (contando desde la derecha y comenzando en $n = 0$), su contribución al valor del número es $a \times 10^n$.

Por ejemplo, la secuencia de dígitos 123 representa el número:

$$N = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 = 123$$

De manera análoga, se pueden definir sistemas posicionales con una base distinta de 10. Cualquier número natural $\beta \geq 2$ puede utilizarse como base. En ese caso, un número natural N se representa como:

$$N = (a_n a_{n-1} \dots a_1 a_0)_\beta = a_n \beta^n + a_{n-1} \beta^{n-1} + \dots + a_1 \beta^1 + a_0 \beta^0, \quad (1)$$

donde $(0 \leq a_i \leq \beta - 1)$.

Como hemos visto, un mismo valor puede expresarse en distintas bases, como decimal, binaria, octal o hexadecimal. En particular, el sistema binario es el que se utiliza internamente en las computadoras.

3. Sistema binario

“Existen 10 tipos de personas: las que entienden binario y las que no.”

Una de las principales ventajas de los sistemas posicionales es que permiten definir reglas generales y sencillas para las operaciones aritméticas. Además, dichas reglas tienden a simplificarse cuanto menor es la base del sistema. Esta observación nos lleva a considerar el sistema de base $\beta = 2$, o **sistema binario**, en el que solo existen los dígitos 0 y 1 (bits).

Sin embargo, existe también una razón fundamental de naturaleza física para su uso. En su nivel más básico, una computadora solo puede registrar dos estados: la presencia o ausencia de flujo eléctrico en una determinada parte del circuito. Estos dos estados pueden asociarse, convencionalmente, a los dígitos 1 (flujo de electricidad) y 0 (ausencia de flujo). Por medio de circuitos adecuados, una computadora puede entonces realizar conteo y operaciones aritméticas utilizando el sistema binario.

3.1. Definición del sistema binario

El **sistema binario** es un caso particular de los sistemas de numeración posicionales (presentados en la Ecuación 1), donde la base es $\beta = 2$. En consecuencia, cada número se expresa como una suma de potencias de 2, ponderadas por coeficientes que solo pueden ser 0 o 1:

$$N = (b_{n-1} \dots b_1 b_0)_2 = b_{n-1} \times 2^{n-1} + \dots + b_1 \times 2^1 + b_0 \times 2^0 \quad (2)$$

Así, por ejemplo, $(1101)_2$ representa el número decimal $(13)_{10}$, ya que:

$$1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13$$

Observación

El subíndice $(\)_\beta$ indica la base del sistema de numeración utilizado. A lo largo de este apunte se explicitará la base para evitar ambigüedades; por ejemplo, $(11)_{10} \neq (11)_2$.

3.2. Representación binaria de enteros sin signo

Un número entero en base binaria puede escribirse de distintas formas equivalentes, ya que los ceros a la izquierda del bit más significativo no alteran

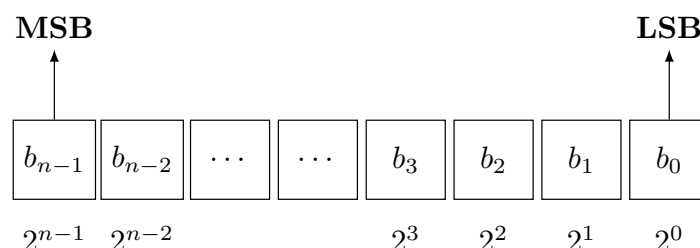
su valor numérico. Por ejemplo, el número 6 puede representarse como $(110)_2$, $(0110)_2$ o $(00000110)_2$.

Sin embargo, esta equivalencia depende del contexto. En notación matemática es habitual omitir los ceros a la izquierda, mientras que en sistemas digitales se utilizan representaciones de longitud fija (por ejemplo, 8, 16 o 32 bits), donde la cantidad de bits está predefinida y los ceros a la izquierda deben conservarse.

Para mejorar la legibilidad, es común agrupar los bits de cuatro en cuatro (o de ocho en ocho) y separarlos con espacios. Por ejemplo, el número 6 en un formato de 8 bits puede escribirse como $(0000\ 0110)_2$.

Al trabajar con números binarios, es frecuente referirse a bits individuales dentro de la representación. Para ello, se asigna un índice a cada posición. En un número de 8 bits, las posiciones se numeran de derecha a izquierda, desde b_0 hasta b_7 .

Sea un número entero representado con n bits, $b_{n-1}b_{n-2} \dots b_1b_0$. El bit más a la derecha (b_0) se denomina *bit menos significativo* (**LSB**, *Least Significant Bit*), ya que tiene el menor peso (2^0). A medida que se avanza hacia la izquierda, el peso de los bits aumenta, siendo el bit más a la izquierda (b_{n-1}) el *bit más significativo* (**MSB**, *Most Significant Bit*), con peso 2^{n-1} .



Desde el punto de vista de la memoria, lo relevante no es la notación utilizada sino el patrón de bits almacenado. Un mismo valor puede escribirse de distintas maneras, pero todas ellas corresponden al mismo patrón binario subyacente (independientemente de su interpretación).

El lenguaje C no proporciona, en sus versiones tradicionales, una notación estándar para literales binarios. Sin embargo, un mismo patrón de bits puede especificarse utilizando notación decimal, hexadecimal, octal o incluso como carácter.

Ejemplo

El patrón de bits 01000001 puede representarse mediante las siguientes expresiones:

```

#include <stdio.h>
int main() {
    char A = 65;           // Decimal
    char B = 0x41;         // Hexadecimal
    char C = 0101;         // Octal
    char D = 0b01000001;   // Binario (C23 o extensión)
    char E = 'A';         // Character
    printf("%d %d %d %d %d\n", A, B, C, D, E);
    return 0;
}

```

Todas estas expresiones generan el mismo valor numérico y, en consecuencia, el mismo patrón de bits en memoria.

3.3. Conversión entre sistemas

3.3.1. Binario a decimal

La conversión de binario a decimal es directa empleando la definición de sistema de numeración posicional en base 2 vista en la Ecuación 2:

$$(b_{n-1} \cdots b_3 b_2 b_1 b_0)_2 = (b_3 \times 2^3 + b_2 \times 2^2 + b_1 \times 2^1 + b_0 \times 2^0)_{10}$$

Ejemplo

Supongamos que queremos convertir el número binario $(01101101)_2$ a decimal:

$$(01101101)_2 = 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^0 = (109)_{10}$$

3.3.2. Decimal a binario

Para convertir un número entero no negativo N de base decimal a base binaria, se utiliza el algoritmo de divisiones sucesivas por 2. En cada paso, se divide el número por 2 y se registra el resto de la división, que necesariamente es 0 o 1.

Los restos obtenidos corresponden a los bits del número en base 2: el primer resto es el bit menos significativo b_0 , el siguiente es b_1 , y así sucesivamente. Por lo tanto, la representación binaria de N se obtiene leyendo

los restos en orden inverso al de su obtención, es decir, desde el último resto hasta el primero.

Este procedimiento garantiza una representación única, ya que corresponde a la expansión de N en base 2.

Ejemplo

Supongamos que queremos convertir el número $(149)_{10}$ a binario. Dividimos sucesivamente por dos hasta que el cociente entero sea 0 y registrando el valor de los restos, tal cual se muestra en el siguiente procedimiento:

$149/2=74$	Resto=1 $\rightarrow b_0=1$
$74/2=37$	Resto=0 $\rightarrow b_1=0$
$37/2=18$	Resto=1 $\rightarrow b_2=1$
$18/2=9$	Resto=0 $\rightarrow b_3=0$
$9/2=4$	Resto=1 $\rightarrow b_4=1$
$4/2=2$	Resto=0 $\rightarrow b_5=0$
$2/2=1$	Resto=0 $\rightarrow b_6=0$
$1/2=0$	Resto=1 $\rightarrow b_7=1$

Notar que el último resto se considera 1 aunque en realidad es menor que 1. Entonces, $(149)_{10} = (1001\ 0101)_2$.

3.4. Rango de enteros sin signo

En la Tabla 1 se muestra la equivalencia entre la representación decimal y binaria para números enteros sin signo de 4 bits. En este caso, pueden representarse valores en el rango de 0 a 15, es decir, un total de $2^4 = 16$ valores distintos.

En general, el rango de valores enteros no negativos representables con n bits está dado por:

$$0 \leq x \leq 2^n - 1$$

donde n es la cantidad de bits utilizados. Esto implica que pueden representarse exactamente 2^n valores distintos.

4. Otras representaciones

Además del sistema binario, existen otros dos sistemas posicionales de interés en el ámbito computacional: el **sistema octal** (base 8) y el **sistema hexadecimal** (base 16).

Tabla 1: Representación de números sin signo en binario.

Representación decimal	Representación en binario
15	1111
14	1110
13	1101
12	1100
11	1011
10	1010
9	1001
8	1000
7	0111
6	0110
5	0101
4	0100
3	0011
2	0010
1	0001
0	0000

4.1. Sistema hexadecimal

“Hexadecimal: el idioma nativo de quienes ya se cansaron de contar bits.”

Un inconveniente importante del sistema binario es su “*verbosidad*”, es decir, la gran cantidad de dígitos necesarios para representar un valor. Por ejemplo, para representar $(158)_{10} = (1001\ 1110)_2$ se requieren ocho cifras, en contraste con las tres necesarias en el sistema decimal. Esto sugiere que la representación decimal es más compacta. Sin embargo, implementar un microprocesador que opere directamente en base decimal sería considerablemente más complejo. Además, si bien es posible convertir entre decimal y binario, el proceso no resulta directa (como se analizó en la Sección 3.3).

El **sistema hexadecimal** (base 16) ofrece una solución eficiente a este problema. Presenta dos ventajas clave:

- Su notación es más compacta.
- La conversión con el sistema binario es directa.

Por estas razones, el sistema hexadecimal se utiliza ampliamente en el ámbito de la computación.

Tabla 2: Equivalencia entre diferentes sistemas de numeración.

Decimal	Octal	Hexadecimal	Binario
0	0	0	0000
1	1	1	0001
2	2	2	0010
3	3	3	0011
4	4	4	0100
5	5	5	0101
6	6	6	0110
7	7	7	0111
8	10	8	1000
9	11	9	1001
10	12	A	1010
11	13	B	1011
12	14	C	1100
13	15	D	1101
14	16	E	1110
15	17	F	1111

Dado que en el sistema hexadecimal la base es 16, cada dígito a la izquierda del punto representa el valor de ese dígito multiplicado por una potencia de 16, dependiendo de su posición. Cada dígito hexadecimal puede tomar uno de 16 valores distintos, que van de 0 hasta $(15)_{10}$. Como el sistema decimal solo cuenta con 10 dígitos, se utilizan seis símbolos adicionales para representar los valores del $(10)_{10}$ al $(15)_{10}$. Estos símbolos son las letras del abecedario de la A a la F. Así se obtiene la tabla de equivalencias entre sistemas, presentada en la Tabla 2.

De esta manera, el número $(15E)_{16}$ representa:

$$(15E)_{16} = 1 \times 16^2 + 5 \times 16^1 + 14 \times 16^0 = (350)_{10} = (0001\ 0101\ 1110)_2,$$

dado que $(E)_{16} = (14)_{10}$.

Como se puede observar el sistema hexadecimal es muy compacto y fácil de leer. Además la conversión entre hexadecimal y binario es muy fácil. La Tabla 2 contiene toda la información necesaria. Para convertir de hexadecimal a binario simplemente hay que sustituir cada dígito hexadecimal por sus correspondientes cuatro bits.

Ejemplo

Para convertir el número $(8AF3)_{16}$ a binario:

HEXADECIMAL	8	A	F	3
BINARIO	1000	1010	1111	0011

Por lo tanto, $(8AF3)_{16} = (1000\ 1010\ 1111\ 0011)_2$.

Para convertir un número de binario a hexadecimal es casi tan fácil. El primer paso es rellenar el número con ceros para asegurarse que el número de bits es múltiplo de cuatro. Luego, separar el número en grupos de cuatro bits y convertir cada grupo utilizando en la Tabla 2.

Ejemplo

Convertir el número binario $(0001\ 0011\ 0010\ 1110)_2$ en hexadecimal:

BINARIO	0001	0011	0010	1110
HEXADECIMAL	1	3	2	E

Por lo tanto, $(0001\ 0011\ 0010\ 1110)_2 = (132E)_{16}$.

4.2. Sistema octal

De manera análoga, se puede trabajar con otras bases. Por ejemplo, otro sistema de representación, aunque menos extendido que el hexadecimal en la práctica actual, es el **sistema octal**, cuya base es 8. Este sistema utiliza ocho cifras: los números enteros del 0 al 7. En la Tabla 2 se muestra la equivalencia entre sistemas. La conversión puede realizarse de forma similar a la ya vista. Cabe destacar que cada dígito en sistema octal puede representarse exactamente con tres bits.

Ejemplo

Convertir el número $(1514)_8$ en formato octal a binario:

OCTAL	1	5	1	4
BINARIO	001	101	001	100

Entonces, $(1514)_8 = (0011\ 0100\ 1100)_2$

Observación

La notación más usual para el formato octal consiste en poner un 0 antes de los dígitos octales. Por ejemplo, $(777)_8 = 0777$.

El sistema octal fue especialmente útil en las primeras etapas de la computación, cuando se trabajaba con una menor cantidad de bits, y aún se utiliza en algunos casos puntuales, como en los sistemas de archivos tipo UNIX.

5. Unidades de información

5.1. Bit

*“Todo en una computadora se reduce a una decisión elemental:
0 o 1.”*

El sistema binario consta únicamente de los dígitos 0 y 1, denominados **bits** (del inglés *binary digits*). En notación binaria, los valores 0 y 1 tienen el mismo significado que en notación decimal:

$$0_2 = 0_{10}, \quad 1_2 = 1_{10}.$$

El bit es la unidad mínima de información en un sistema binario. Dado que solo puede representar dos valores distintos (cero o uno), podría parecer que tiene poca utilidad. Sin embargo, puede emplearse para representar una gran variedad de conceptos: verdadero o falso, encendido o apagado, presente o ausente, entre otros. Todo depende de cómo se defina la estructura de datos. Por ejemplo, podemos establecer que un bit con valor cero represente “falso” y un bit con valor uno represente “verdadero”.

5.2. Nibble

Un **nibble** es un conjunto de cuatro bits. No se trata de una estructura de datos particularmente destacada, salvo en dos casos relevantes: la representación de números en formato BCD (*binary coded decimal*) y en formato hexadecimal¹.

Con un nibble se pueden representar $2^4 = 16$ valores distintos. En el caso del sistema hexadecimal, estos valores son: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B,

¹El sistema hexadecimal se analiza en detalle en la Sección 4.1

C, D, E y F. En formato BCD se utilizan solo diez dígitos (del 0 al 9), pero también se requiere un conjunto de cuatro bits para codificarlos.

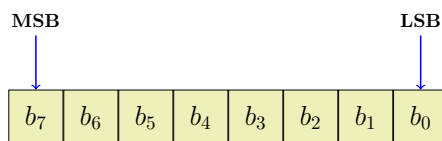
En resumen, un nibble (4 bits) permite representar un dígito en formato BCD o un dígito en formato hexadecimal.

5.3. Byte

Un **byte** es una unidad de información digital compuesta por 8 bits. Es la unidad básica de almacenamiento en la mayoría de las arquitecturas de computadoras y se utiliza para representar caracteres, números y otros tipos de datos. La memoria principal está direccionada por bytes, lo que significa que el elemento más pequeño al que se puede acceder de forma individual es un valor de 8 bits.

Para acceder a una porción menor de información —como un solo bit o un conjunto de pocos bits— es necesario leer el byte completo que los contiene y aplicar una operación de enmascaramiento² para descartar los bits no deseados.

Los bits en un byte se pueden numerar desde cero hasta siete usando la siguiente convención:



El bit 0 (b_0) se denomina bit de menor orden o bit menos significativo (LSB, por la sigla en inglés *Least Significant Bit*), mientras que el bit 7 (b_7) es el bit de mayor orden o bit más significativo (MSB, por *Most Significant Bit*).

Cabe destacar que un byte contiene exactamente dos nibbles. Los bits del 0 al 3 conforman el *low-order nibble*, mientras que los bits del 4 al 7 forman el *high-order nibble*. Como cada nibble puede representarse con un dígito hexadecimal, se necesitan dos dígitos hexadecimales para representar un byte completo.

²En informática, se denomina “*máscara*” al conjunto de datos que, junto con una operación determinada, permite extraer selectivamente ciertos bits almacenados en otro conjunto. Abordaremos este concepto con mayor detalle en el próximo apunte.

5.4. Palabra

En informática, una **palabra** (o *word*) es una unidad de datos cuyo tamaño está determinado por la arquitectura del procesador. En general, su longitud coincide con el tamaño de los registros de propósito general de la CPU.

Típicamente, una palabra puede tener 16, 32 o 64 bits, según la arquitectura. Este tamaño se utiliza como unidad natural para las operaciones aritméticas y lógicas, así como para los accesos a memoria más eficientes.

Una palabra puede definirse, entonces, como un conjunto de n bits que la máquina maneja como una unidad. Sus bits se numeran desde 0 (b_0) hasta $n - 1$ (b_{n-1}). Al igual que en el caso del byte, b_0 corresponde al bit menos significativo, mientras que b_{n-1} corresponde al más significativo.

Por ejemplo, una palabra de 16 bits contiene dos bytes, y por lo tanto, cuatro nibbles, como se ilustra en la siguiente figura:

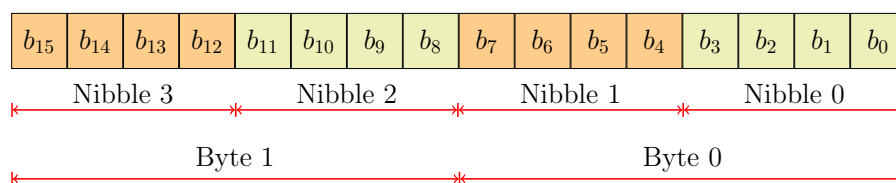


Figura 1: Bytes y nibbles en una palabra de 16 bits.

Con n bits se pueden representar 2^n valores diferentes. Por ejemplo, en las arquitecturas modernas que utilizan palabras de 64 bits, es posible representar hasta 2^{64} valores distintos.

Uno de los principales usos de las palabras es la representación de valores enteros. En el caso de valores sin signo (*unsigned*), la representación se realiza directamente a partir del valor binario de los bits. Para representar números con signo existen diferentes enfoques; uno de los más utilizados es el método del *complemento a dos*, que se abordará en la Sección 8.2.

5.5. Múltiplos del byte

En la práctica, las cantidades de información se expresan utilizando múltiplos del byte. Existen dos convenciones principales para definir estos múltiplos: una basada en potencias de 10 (sistema decimal) y otra basada en potencias de 2 (sistema binario).

Sistema decimal (SI) Se utiliza principalmente en dispositivos de almacenamiento (por ejemplo, discos rígidos). Los múltiplos se definen como:

$$1 \text{ kB} = 10^3 \text{ B}, \quad 1 \text{ MB} = 10^6 \text{ B}, \quad 1 \text{ GB} = 10^9 \text{ B}, \quad 1 \text{ TB} = 10^{12} \text{ B}.$$

Sistema binario (IEC) Se utiliza habitualmente en memoria principal y en contextos donde la organización es en base 2. Los múltiplos se definen como:

$$1 \text{ KiB} = 2^{10} \text{ B} = 1024 \text{ B}, \quad 1 \text{ MiB} = 2^{20} \text{ B}, \quad 1 \text{ GiB} = 2^{30} \text{ B}, \quad 1 \text{ TiB} = 2^{40} \text{ B}.$$

Observación

Es importante distinguir ambas convenciones, ya que el uso de los prefijos puede variar según el contexto. En particular, algunos sistemas utilizan los prefijos del sistema decimal (kB, MB, GB) para referirse a cantidades binarias, lo que puede generar ambigüedad.

6. Operaciones en sistema binario

6.1. Suma

En binario, la suma de bits tiene la siguiente forma:

$$\begin{aligned} 0 + 0 &= 0 \\ 0 + 1 &= 1 \\ 1 + 0 &= 1 \\ 1 + 1 &= \mathbf{10} \end{aligned}$$

Note que al sumar $1 + 1$ es $(10)_2$, es decir, llevamos 1 a la siguiente posición de la izquierda (acarreo). En el sistema decimal esto es equivalente a sumar, por ejemplo, $8 + 7$ que resulta 15. Por lo tanto, el resultado es un 5 en la posición que estamos sumando y un 1 de acarreo en la siguiente posición a la izquierda.

La siguiente tabla muestra la operación de suma binaria considerando dos operandos (a_i y b_i) y el acarreo de entrada de la posición anterior (c_{in}). Se presentan los resultados de la suma (s_i) y el acarreo de salida (c_{out}), que se propaga al siguiente bit en sumas de múltiples bits:

a_i	b_i	c_{in}	$a_i + b_i + c_{in}$	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Ejemplo

Dados números $A = (0101)_2$ y $B = (0011)_2$, realizar la suma $A + B$:

$$\begin{array}{r}
 \mathbf{1} \quad \mathbf{1} \quad \mathbf{1} \\
 0 \quad 1 \quad 0 \quad 1 \\
 + \quad 0 \quad 0 \quad 1 \quad 1 \\
 \hline
 1 \quad 0 \quad 0 \quad 0
 \end{array}$$

En sistema decimal sería $A + B = 5 + 3 = 8$.

6.2. Resta

La tabla de la resta toma la siguiente forma:

$$\begin{array}{rcl}
 0 - 0 = & 0 \\
 1 - 0 = & 1 \\
 1 - 1 = & 0 \\
 0 - 1 = & \mathbf{11}
 \end{array}$$

La resta $0 - 1$ se resuelve igual que en el sistema decimal, tomando una unidad prestada de la posición siguiente: $0 - 1 = 1$ y “*me llevo*” 1.

La siguiente tabla muestra la operación de resta binaria considerando dos operandos (a_i y b_i) y un préstamo de entrada (c_{in}). Se presentan los resultados de la diferencia (d_i) y el préstamo de salida (c_{out}), que indica si es necesario tomar un préstamo en la siguiente posición en restas de múltiples bits.

a_i	b_i	c_{in}	$a_i - b_i - c_{in}$	c_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Ejemplo

Dados los números $A = (1001)_2$ y $B = (0010)_2$, realizar la resta $A - B$:

$$\begin{array}{r} \\ \\ \\ - \\ \hline \end{array}$$

En sistema decimal sería $A - B = 9 - 2 = 7$. Otra forma mucho más práctica de realizar las restas es utilizar el *complemento a dos* o el *complemento a uno*. Este tema se desarrolla en la Sección 8.2.

6.3. Multiplicación

La tabla de multiplicación binaria toma la siguiente forma:

$$\begin{array}{l} 0 \times 0 = 0 \\ 0 \times 1 = 0 \\ 1 \times 0 = 0 \\ 1 \times 1 = 1 \end{array}$$

La multiplicación de números en binario es muy sencilla dado que cero por cualquier número es cero y el uno es el elemento neutro de la multiplicación.

Ejemplo

Dados los números $A = (1100)_2$ y $B = (0110)_2$, realizar la multiplicación $A \times B$:

$$\begin{array}{rcccc}
 & & 1 & 1 & 0 & 0 \\
 & \times & 0 & 1 & 1 & 0 \\
 \hline
 & & 0 & 0 & 0 & 0 \\
 \\
 & \mathbf{1} & 1 & 1 & 0 & 0 \\
 \mathbf{1} & 1 & 1 & 0 & 0 & \\
 0 & 0 & 0 & 0 & & \\
 \hline
 1 & 0 & 0 & 1 & 0 & 0 & 0
 \end{array}$$

En sistema decimal sería $A \times B = 12 \times 6 = 72$.

$$\begin{array}{ccccccc}
 & & & & 1 & 1 & 0 & 0 \\
 & & & \times & 0 & 1 & 1 & 0 \\
 & & & & \hline
 & & & & 0 & 0 & 0 & 0 \\
 & \mathbf{1} & 1 & 1 & 0 & 0 & & \\
 \mathbf{1} & 1 & 1 & 0 & 0 & & & \\
 0 & 0 & 0 & 0 & & & & \\
 \hline
 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0
 \end{array}$$

En sistema decimal sería $A \times B = 12 \times 6 = 72$.

6.4. Bandera de acarreo (*Carry Flag*)

Al operar con valores en representación computacional, es fundamental analizar el estado de las banderas. Una **bandera** es un bit especial que se utiliza para indicar el resultado de ciertas operaciones o para controlar el comportamiento del procesador. Las banderas forman parte de un registro denominado **registro de estado** o **registro de banderas** (*status register* o *flags register*)³, el cual es actualizado automáticamente por el procesador tras la ejecución de instrucciones aritméticas, lógicas, de comparación, entre otras.

La bandera de **acarreo** (*Carry Flag*, CF) se activa cuando ocurre un acarreo en una suma o un préstamo en una resta, lo que indica que se ha excedido el rango representable para números sin signo.

Ejemplos

A continuación se presentan ejemplos que trabajan con números sin signo representados con 8 bits:

- Supongamos que realizamos la suma $255 + 1$:

$$\begin{array}{cccccccc}
 & \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{1} & & \mathbf{1} & \mathbf{1} & \mathbf{1} \\
 & 1 & 1 & 1 & 1 & & 1 & 1 & 1 & 1 \\
 + & 0 & 0 & 0 & 0 & & 0 & 0 & 0 & 1 \\
 \mathbf{1} & \hline
 & 0 & 0 & 0 & 0 & & 0 & 0 & 0 & 0
 \end{array}$$

³En arquitecturas x86-64, el registro `rflags` almacena indicadores de estado y control del procesador. Contiene bits que reflejan el resultado de operaciones aritméticas (como el *carry* o el *zero flag*) y configuraciones que afectan el comportamiento de ejecución. El registro `rflags` se estudiará en detalle en el apunte **Arquitectura x86-64 y Lenguaje Ensamblador**.

La bandera *carry* queda en $CF=1$ debido a que se produjo un acarreo al sumar los bits más significativos. El resultado es incorrecto si se interpretan los operandos como números *unsigned*, ya que $255 + 1 \neq 0$. Sin embargo, el resultado es correcto si se los interpreta como números *signed*, dado que $(-1) + 1 = 0$.

- Supongamos que realizamos la resta $0 - 1$:

$$\begin{array}{r}
 \quad \quad \quad \begin{array}{cccc} 1 & 1 & 1 & 1 \end{array} \quad \begin{array}{ccc} 1 & 1 & 1 \end{array} \\
 \quad \quad \quad \begin{array}{cccc} 0 & 0 & 0 & 0 \end{array} \quad \begin{array}{cccc} 0 & 0 & 0 & 0 \end{array} \\
 - \quad \quad \quad \begin{array}{cccc} 0 & 0 & 0 & 0 \end{array} \quad \begin{array}{cccc} 0 & 0 & 0 & 1 \end{array} \\
 \hline
 \begin{array}{cccc} 1 & 1 & 1 & 1 \end{array} \quad \begin{array}{cccc} 1 & 1 & 1 & 1 \end{array}
 \end{array}$$

La bandera *carry* queda en $CF=1$ porque se produjo un préstamo en la última posición. El resultado es incorrecto si se interpretan los operandos como números *unsigned*, ya que $0 - 1 \neq 255$. Sin embargo, es correcto si se los interpreta como números *signed*, dado que $0 - 1 = -1$.

Conclusión

Si la suma de dos números enteros en binario genera un acarreo en el bit más significativo, o si la resta requiere un préstamo en ese mismo bit, se activa la bandera *carry* ($CF=1$). Esto indica que el resultado es incorrecto si se interpretan los operandos como números *unsigned*, ya que queda fuera del rango representable para números con signo con la cantidad de bits disponible. En consecuencia, sería necesario utilizar más bits para almacenar el resultado correctamente.

7. Operaciones en sistema hexadecimal

A la hora de realizar operaciones en sistema hexadecimal se puede optar por hacer un cambio de sistema como paso intermedio o bien operar directamente en hexadecimal. Veamos la suma y la resta.

7.1. Suma

La suma se realiza de manera análoga a lo visto teniendo en cuenta el acarreo correspondiente.

Ejemplo

$$\begin{array}{r}
 \mathbf{1} \\
 1 \quad \text{A} \quad \text{E} \\
 + \quad \quad 1 \quad 8 \\
 \hline
 1 \quad \text{C} \quad 6
 \end{array}$$

7.2. Resta

Para realizar la resta, podemos proceder restando dígito a dígito y teniendo en cuenta los acarreos.

Ejemplo

$R = N_1 - N_2$, con $N_1 = (1A6)_{16}$ y $N_2 = (1C)_{16}$.

$$\begin{array}{r}
 \mathbf{1} \\
 1 \quad \text{A} \quad 6 \\
 - \quad \quad 1 \quad \text{C} \\
 \hline
 1 \quad 8 \quad \text{A}
 \end{array}$$

Otra forma es calcular el complemento a la base ($C_{16}(N)$) del sustraendo y luego sumarlo al minuendo. Previamente, hay que igualar la cantidad de dígitos. En el ejemplo anterior primero se calcularía el complemento a 16 de N_2 . Para ello recordar la relación entre el complemento a la base y el complemento a la base menos uno, dado que es más simple calcular $C_{15}(N)$:

$$\begin{aligned}
 C_{16}(N) &= C_{15}(N) + 1 \\
 C_{16}(N) &= (\text{FE}3)_{16} + 1 = (\text{FE}4)_{16}
 \end{aligned}$$

Entonces,

$$\begin{array}{r}
 \quad \quad 1 \quad \text{A} \quad 6 \\
 + \quad \quad \text{F} \quad \text{E} \quad 4 \\
 \mathbf{1} \quad \quad 1 \quad 8 \quad \text{A} \\
 \hline
 \end{array}$$

El resultado es correcto ignorando el primer uno.

Observación

Si bien hemos mantenido la notación $()_{16}$ para mantener la coherencia con lo anterior, en realidad la notación más usual para el formato hexadecimal consiste en poner **0x** antes de los dígitos hexadecimales. Por ejemplo, $(\text{ffe8})_{16} = \text{0xffe8}$.

Observación

Si bien el sistema hexadecimal resulta más práctico que el binario para representar e ingresar información en sistemas computacionales —por su notación más compacta y legible para personas—, el sistema binario es más adecuado para las operaciones internas de la computadora. De hecho, todas las computadoras modernas operan en binario a nivel de hardware, ya que los circuitos electrónicos trabajan naturalmente con dos estados (ausencia o presencia de voltaje), lo que se corresponde con los valores 0 y 1.

8. Representación de enteros con signo

“Toda información en una computadora es, en última instancia, una interpretación de bits.”

Hasta ahora hemos trabajado con números sin signo, donde todos son positivos. Sin embargo, también es necesario representar números negativos. Una opción natural es seguir la analogía con el sistema decimal, en el que un número negativo se expresa como su magnitud precedida por el signo “-”. Este método se conoce como formato **magnitud y signo**.

8.1. Representación magnitud y signo

En el formato **magnitud y signo**, se utiliza un bit, generalmente el más significativo, para representar el signo, mientras que los bits restantes indican la magnitud (valor absoluto) del número:

$$(b_{n-1}b_{n-2} \cdots b_1b_0)_2 = (-1)^s \times (b_{n-2} \times 2^{n-2} + \cdots + b_1 \times 2^1 + b_0 \times 2^0),$$

donde $s = b_{n-1}$. De este modo, si el bit más significativo es “1”, el número es negativo y si es “0”, el número es positivo.

Por ejemplo, para representar el número $(-28)_{10}$ primero se convierte el número $(28)_{10}$ a binario, esto es $(0001\ 1100)_2$ y luego se indica que es negativo haciendo uno el bit más significativo: $(-28)_{10} = (1001\ 1100)_2$.

$$(1001\ 1100)_2 = (-1)^1 \times (2^4 + 2^3 + 2^2) = (-28)_{10}$$

En la Tabla 3 se muestra la representación de números con signo utilizando números de 4 bits. El rango de valores representables es:

$$\boxed{-2^{n-1} + 1 \leq x \leq 2^{n-1} - 1}$$

donde n es la cantidad de bits utilizados para representar el número.

El enfoque magnitud y signo presenta dos desventajas importantes. En primer lugar, existe una doble representación del cero, ya que tanto $(1000)_2$ como $(0000)_2$ representan el mismo valor, lo que complica una operación fundamental: la comparación con cero. Por otro lado, las operaciones aritméticas resultan más complejas. Por ejemplo, para realizar una suma, primero es necesario determinar si los números tienen el mismo signo y, en caso afirmativo, sumar sus magnitudes. Si los signos son diferentes, se debe restar el menor del mayor y asignar el signo del número con mayor valor absoluto.

Ejemplo

Supongamos que se utilizan 4 bits para representar números en magnitud y signo. Ahora sumemos $(+3) + (-3)$:

$$\begin{array}{rcccccl}
 & 0 & 0 & 1 & 1 & (+3) \\
 + & 1 & 0 & 1 & 1 & (-3) \\
 \hline
 & 1 & 1 & 1 & 0 & (-6) \quad \textcolor{red}{\times}
 \end{array}$$

Esperábamos obtener cero, pero el resultado es -6, que es incorrecto.

Debido a estas limitaciones, este enfoque tuvo un uso limitado. A continuación, exploraremos alternativas más eficientes basadas en el **complemento del número**.

8.2. Complementos a la base y a la base menos uno

Otro enfoque para representar números con signo se basa en los operadores complemento. Estos operadores se utilizan ampliamente para representar números negativos. Existen dos operadores complemento: el **complemento a la base** y el **complemento a la base menos uno**. Es decir, para los

Tabla 3: Representación de números con signo en magnitud y signo.

Representación decimal con signo	Representación magnitud y signo
7	0111
6	0110
5	0101
4	0100
3	0011
2	0010
1	0001
0	0000
0	1000
-1	1001
-2	1010
-3	1011
-4	1100
-5	1101
-6	1110
-7	1111

números binarios (donde la base es 2), existen los complementos a 2 y a 1. En base octal serían complemento a 8 y a 7. En el sistema decimal, complemento a 10 y a 9, y así sucesivamente.

Desde el punto de vista computacional, nos interesa especialmente el **complemento a dos**, ya que es la representación más utilizada para el manejo de números con signo en sistemas digitales. Esta técnica permite realizar operaciones aritméticas de manera eficiente, simplificando la implementación del hardware y optimizando el procesamiento de datos.

8.3. Representación en complemento a dos

La representación en **complemento a dos** es un sistema posicional para números enteros en el que los positivos se expresan en binario natural, mientras que los negativos se obtienen aplicando el operador complemento a dos.

El **operador complemento a dos** de un número N se define como:

$$C_2(N) = 2^n - N$$

donde n es la cantidad de bits que utilizamos para representar el número.

Ejemplo

Supongamos que tenemos el número $N = 84$. Su representación en binario utilizando 8 bits es $(01010100)_2$. Aplicando el operador complemento a dos según la definición de la ecuación anterior:

$$C_2(N) = (100000000)_2 - (01010100)_2 = (10101100)_2$$

Operador complemento a dos: método alternativo

El cálculo del complemento a dos puede realizarse de manera muy sencilla mediante un método alternativo. En efecto, se puede ver que para calcular el complemento a 2 de un número binario sólo basta con revisar todos los dígitos desde el menos significativo hacia el más significativo y mientras se consiga un cero dejarlo igual. Al conseguir el primer número 1, dejarlo igual para luego cambiar el resto de ellos hasta llegar al más significativo. Así podemos decir rápidamente que el complemento a 2 de $(1010\ 0000)_2$ es $(0110\ 0000)_2$, que el complemento a 2 de $(1111)_2$ es $(0001)_2$, etc.

Observación

Notar que $C_2(C_2(N)) = N$. Es decir, si a un número N le aplicamos el operador complemento a dos y al resultado le volvemos a aplicar el mismo operador obtenemos el número original.

Por lo tanto, para un N su representación en binario complemento a dos es la siguiente:

- **Representación de números positivos:** Los números positivos se representan de forma estándar en binario, igual que en el sistema binario sin signo. El bit más significativo (MSB) es 0, indicando que el número es positivo.
- **Representación de números negativos:** Para obtener la representación de un número negativo en complemento a dos, primero se toma el valor absoluto del número y se representa en binario. Luego, se aplica el operador complemento a dos al valor absoluto. El bit más significativo (MSB) es 1, indicando que el número es negativo.

Ejemplos

- Si tenemos el número $N_1 = (84)_{10}$ y lo representamos con 8 bits, dado que N_1 es un número positivo, utilizamos la representación en binario natural:

$$(84)_{10} = 2^6 + 2^4 + 2^2$$

Por lo tanto:

$$N_1 = (0101\ 0100)_2$$

- Si tenemos el número $N_2 = (-84)_{10}$ y lo representamos con 8 bits, dado que N_2 es un número negativo, aplicamos la definición de complemento a dos:

$$N_2 = C_2(|N_2|) = \underbrace{(1\ 0000\ 0000)_2}_{2^8} - \underbrace{(0101\ 0100)_2}_{84} = (1010\ 1100)_2$$

Por lo tanto:

$$(-84)_{10} = (1010\ 1100)_2$$

- Si queremos hacer la operación $N_1 + N_2 = 84 + (-84) = 0$, resulta

$$(0101\ 0100)_2 + (1010\ 1100)_2 = (\mathbf{1}\ 0000\ 0000)_2$$

El resultado es correcto si despreciamos el acarreo.

Conclusión:

Aquí se observa una propiedad fundamental de la representación en complemento a dos: la suma se realiza de manera uniforme para operandos con y sin signo, utilizando el mismo circuito que para enteros sin signo.

En particular, el resultado de la suma queda representado en n bits, por lo que cualquier acarreo que exceda ese tamaño se descarta automáticamente. En este caso, el resultado es $(0000\ 0000)_2$, que corresponde correctamente a 0 en decimal.

Esta propiedad permite simplificar el diseño del hardware aritmético, ya que no es necesario un tratamiento especial para números negativos.

Observaciones

- Notar que, al trabajar en binario complemento a dos (es decir, con números con signo), si el bit más significativo es uno, el número es negativo; mientras que si es cero, el número es positivo:

$$\begin{aligned}N_1 &= (84)_{10} = (0101\ 0100)_2 \\N_2 &= (-84)_{10} = (1010\ 1100)_2\end{aligned}$$

- La secuencia de bits del número $N_2 = (1010\ 1100)_2$ también se puede interpretar como $N_2 = -2^7 + 2^5 + 2^3 + 2^2 = -84$.

En efecto, un número binario de n bits en complemento a dos se puede interpretar de la siguiente manera:

$$N = (b_{n-1}b_{n-2} \dots b_1b_0)_2 = -b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i$$

donde:

- b_{n-1} = bit más significativo (MSB).
- b_i = bit en la posición i (contando desde la derecha, empezando en 0).

La diferencia con binario sin signo es que el MSB tiene peso negativo.

De la primera observación se obtiene el método para convertir un número binario en complemento a dos a su equivalente decimal:

1. Si el bit más significativo (MSB) es 0, el número es positivo y su valor en decimal se obtiene directamente convirtiendo el número binario a decimal.
2. Si el bit más significativo (MSB) es 1, el número es negativo y se debe aplicar el complemento a dos para obtener su valor absoluto. Es decir, aplicar el complemento a dos y agregar el signo negativo.

De la segunda observación surge una forma alternativa de convertir un número binario en complemento a dos a su equivalente decimal.

En la Tabla 4 se muestra la representación de números con signo utilizando enteros de 4 bits.

Tabla 4: Representación de números con signo en complemento a dos.

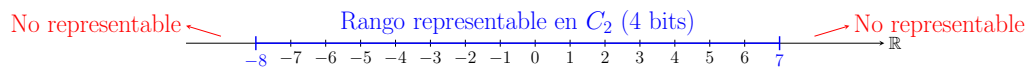
Decimal	Complemento a dos
7	0111
6	0110
5	0101
4	0100
3	0011
2	0010
1	0001
0	0000
-1	1111
-2	1110
-3	1101
-4	1100
-5	1011
-6	1010
-7	1001
-8	1000

El **rango** de valores representables utilizando complemento a dos con n bits es:

$$-2^{n-1} \leq x \leq 2^{n-1} - 1$$

En total, pueden representarse 2^n valores distintos. De ellos, 2^{n-1} corresponden a valores no negativos (incluyendo el cero) y 2^{n-1} a valores negativos.

Por ejemplo, con cuatro bits el rango representable abarca todos los números enteros desde -8 hasta 7 , como se muestra en la siguiente figura:



Si necesitamos representar un rango mayor debemos aumentar la cantidad de bits. La cuestión del rango es crucial al momento de realizar operaciones, dado que el resultado puede quedar fuera de rango, dando un resultado incorrecto.

Observación

Es importante notar que dada una secuencia de n bits, esa secuencia de bits puede ser interpretada como un número sin signo en el rango $[0 : 2^n - 1]$ utilizando la definición de binario natural o un número con signo en el rango $[-2^{n-1} : 2^{n-1} - 1]$ utilizando la representación en **complemento a dos**. La siguiente tabla muestra esta diferencia en la interpretación para números con 8 bits de acuerdo lo visto previamente.

Representación binaria	Números <i>unsigned</i>	Números <i>signed</i> complemento a dos
0000 0000	0	0
0000 0001	1	+1
⋮	⋮	⋮
0111 1111	127	+127
1000 0000	128	-128
⋮	⋮	⋮
1111 1110	254	-2
1111 1111	255	-1

8.4. Representación en complemento a uno

La representación binaria en **complemento a uno** es un sistema de representación posicional para números enteros en el que los positivos se representan en binario natural y los negativos se representan utilizando el operador complemento a uno.

El **operador complemento a uno** de un número N se define como:

$$C_1(N) = 2^n - N - 1$$

donde n es la cantidad de bits que se utiliza para la representación del número.

Ejemplo

Supongamos que tenemos el número $N = 84$. Su representación en binario es utilizando 8 bits es $(01010100)_2$. Aplicando el operador complemento a uno según la definición de la ecuación anterior:

$$C_1(N) = (100000000)_2 - (01010100)_2 - 1 = (10101011)_2$$

Operador complemento a uno: método alternativo

Observando el ejemplo anterior, vemos que una forma sencilla de hallar el complemento a uno de un número binario es invirtiendo todos sus dígitos. Este método es fácil de implementar mediante puertas lógicas.

Observaciones

- Notar que $C_1(C_1(N)) = N$. Es decir, si a un número N le aplicamos el operador complemento a uno y al resultado le volvemos a aplicar el mismo operador obtenemos el número original.
- Notar que el complemento a dos se puede obtener también tomando el complemento a uno de un número y luego sumando 1 al resultado: $C_2(N) = C_1(N) + 1$.

Por lo tanto, para un N su representación en binario complemento a uno es la siguiente:

- **Representación de números positivos:** Los números positivos se representan de forma estándar en binario, igual que en el sistema binario sin signo. El bit más significativo (MSB) es 0, indicando que el número es positivo.
- **Representación de números negativos:** Para obtener la representación de un número negativo en complemento a uno, primero se toma el valor absoluto del número y se representa en binario. Luego, se aplica el operador complemento a uno al valor absoluto. El bit más significativo (MSB) es 1, indicando que el número es negativo.

Ejemplos

- Si tenemos el número $N_1 = (84)_{10}$ y lo representamos con 8 bits, dado que N_1 es un número positivo, utilizamos la representación en binario natural:

$$(84)_{10} = 2^6 + 2^4 + 2^2$$

Por lo tanto:

$$N_1 = (0101\ 0100)_2$$

- Si tenemos el número $N_2 = (-84)_{10}$ y lo representamos con 8 bits, dado que N_2 es un número negativo, aplicamos la definición de complemento a uno:

$$N_2 = \underbrace{(1\ 0000\ 0000)_2}_{2^8} - \underbrace{(0101\ 0100)_2}_{84} - 1 = (1010\ 1011)_2$$

Por lo tanto:

$$N_2 = (-84)_{10} = (1010\ 1011)_2$$

- Si queremos hacer la operación $N_1 + N_2 = 84 + (-84) = 0$, resulta

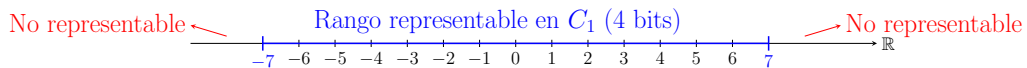
$$(0101\ 0100)_2 + (1010\ 1011)_2 = (1111\ 1111)_2$$

¿El resultado es correcto? Sí, en complemento a uno el resultado es correcto.

El rango de valores decimales utilizando el complemento a uno para n bits será:

$$\boxed{-2^{n-1} + 1 \leq x \leq 2^{n-1} - 1}$$

El total de números positivos (incluyendo el cero) será 2^{n-1} y el de negativos $2^{n-1} - 1$. Por ejemplo, con cuatro bits el rango representable abarca todos los números enteros desde -7 hasta 7 , como se muestra en la siguiente figura:



En la Tabla 5 se presenta, de manera comparativa, la representación de números con signo utilizando enteros de 4 bits para los tres sistemas de representación estudiados. Observar que tanto la representación en complemento a

uno como la representación en magnitud y signo tienen dos representaciones posibles del cero, lo cual es una desventaja. Notar también que el bit más significativo indica el signo de la representación del número en decimal con signo en cualquiera de las representaciones.

Tabla 5: Representación de números con signo con diferentes enfoques.

Decimal	Complemento a dos	Complemento a uno	Magnitud y signo
7	0111	0111	0111
6	0110	0110	0110
5	0101	0101	0101
4	0100	0100	0100
3	0011	0011	0011
2	0010	0010	0010
1	0001	0001	0001
0	0000	0000	0000
0	0000	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	1011	1100
-5	1011	1010	1101
-6	1010	1001	1110
-7	1001	1000	1111
-8	1000		

Observaciones

- Se puede observar que para conseguir el complemento a uno de un número binario tan solo basta con invertir todos los dígitos (esto quiere decir cambiar 0 por 1 y viceversa). Este método es muy simple y es el que habitualmente se realiza.
- Se observa además que hay doble representación del cero. Es decir, tanto $(0 \cdots 00)_2$ como $(1 \cdots 11)_2$ representan al cero.
- Las operaciones con números en complemento uno tienen inconvenientes. Supongamos que queremos hacer la operación $(+3) + (-2)$ en complemento a uno utilizando 4 bits:

$$\begin{array}{rcccccl}
 & 0 & 0 & 1 & 1 & (+3) \\
 + & 1 & 1 & 0 & 1 & (-2) \\
 \hline
 1 & 0 & 0 & 0 & 0 & (0) \quad \textcolor{red}{\times}
 \end{array}$$

Esperábamos obtener uno, pero el resultado es cero, que es incorrecto. Notemos que, si le sumamos el uno correspondiente al acarreo, obtenemos el resultado correcto:

$$\begin{array}{rcccccl}
 & 0 & 0 & 0 & 0 & (0) \\
 + & 0 & 0 & 0 & 1 & (1) \\
 \hline
 & 0 & 0 & 0 & 1 & (1) \quad \textcolor{green}{\checkmark}
 \end{array}$$

- Estas dos últimas observaciones justifican la preferencia por la representación en complemento a dos frente al complemento a uno para realizar operaciones aritméticas.

9. Operaciones en complemento a dos

“Complemento a dos: la elegancia de hacer restas sumando.”

9.1. Suma

Las operaciones en complemento a dos permiten realizar sumas y restas de números con signo de manera eficiente en sistemas digitales. A continuación, se presentan algunos ejemplos para operar en complemento a dos.

Ejemplos

Sean dos números A y B representados en complemento en binario complemento a dos con 8 bits. Si deseamos obtener la suma $S = A + B$, analizaremos los distintos casos posibles.

1. $A > 0$ y $B > 0$:

$$A = 5, B = 12$$

En binario complemento a dos:

$$\begin{array}{r}
 \\
 \\
+ \\
\hline

\end{array}$$

$S = A + B = 17$ y acarreo $\mathbf{c} = 0$.

2. $\mathbf{A} < \mathbf{0}$ y $\mathbf{B} < \mathbf{0}$:

$$A = -5, B = -12$$

En binario complemento a dos:

$$\begin{array}{r}
 \\
 \\
+ \\
\hline

\end{array}$$

$S = A + B = -17$ y acarreo $\mathbf{c} = 1$.

3. $\mathbf{A} < \mathbf{0}$, $\mathbf{B} > \mathbf{0}$ y $|\mathbf{A}| < |\mathbf{B}|$:

$$A = -5, B = 12$$

En binario complemento a dos:

$$\begin{array}{r}
 \\
 \\
+ \\
\hline

\end{array}$$

$S = A + B = 7$ y acarreo $\mathbf{c} = 1$.

4. $\mathbf{A} > \mathbf{0}$, $\mathbf{B} < \mathbf{0}$ y $|\mathbf{A}| < |\mathbf{B}|$:

$$A = 5, B = -12$$

En binario complemento a dos:

$$\begin{array}{r}
 \\
 \\
+ \\
\hline

\end{array}$$

$S = A + B = -7$ y acarreo $\mathbf{c} = 0$.

En todos los casos, el resultado es correcto si se ignora el acarreo.

9.2. Resta

La operación de resta en complemento a dos se puede presentar como una suma, evitando la necesidad de un circuito específico para la sustracción. Para realizar la operación resta $A - B$ se sigue el siguiente procedimiento:

- Obtener el complemento a dos de B .
- Sumar el complemento a dos de B a A .

Este método unifica las operaciones aritméticas, permitiendo que la resta se implemente con el mismo circuito que la suma. A continuación, se presentan un ejemplo ilustrativo para comprender su aplicación.

Ejemplo

Realizar la operación $84 - 37$ en complemento a dos.

- Representar los números en binario con 8 bits:
 - $(84)_{10} = (0101\ 0100)_2$
 - $(37)_{10} = (0010\ 0101)_2$
- Obtener el complemento a dos de 37:

$$C_2(-37) = (1101\ 1011)_2$$

- Sumar 84 y -37 en binario:

$$(0101\ 0100)_2 + (1101\ 1011)_2 = (1\ 0011\ 1111)_2$$

- Descartar el acarreo más significativo.

El resultado es $(0011\ 1111)_2$, que en decimal equivale a 47, el resultado esperado de $84 - 37$.

Este ejemplo muestra cómo la resta se convierte en una suma utilizando el complemento a dos, permitiendo una implementación eficiente en hardware.

Observaciones

- El complemento a dos es el sistema utilizado en computadoras modernas para la representación de los números con signo, debido a las ventajas al realizar operaciones como hemos visto en los ejemplo anteriores.
- Es importante notar que lo anterior es cierto siempre en cuando nos mantengamos dentro del rango de números representables para la cantidad de dígitos con los que estamos trabajando. Sin embargo, al realizar una operación es posible que nos salgamos de rango, con lo cual el resultado obtenido es erróneo. Por lo tanto, debemos tener en cuenta las banderas de acarreo y desbordamiento como veremos a continuación.

9.3. Bandera de desbordamiento (*Overflow Flag*)

El registro de banderas contiene diversas banderas, entre ellas la *carry flag* (CF) y la *overflow flag* (OF). La *carry flag* ya se presentó en la Sección 6.4. Es importante considerarlas —y no confundirlas—, ya que la ALU (*Arithmetic Logic Unit*) no distingue si el programador está trabajando con aritmética con signo (*signed*) o sin signo (*unsigned*). La ALU simplemente realiza la operación binaria bit a bit y actualiza las banderas correspondientes. Corresponde al programador interpretar el valor de estas banderas luego de la operación. A continuación, analizaremos las dos banderas principales del registro de banderas, que inciden directamente en la ejecución de operaciones aritméticas.

La bandera de **desbordamiento** (*Overflow Flag*, OF) se activa cuando el resultado de una operación aritmética excede el rango representable para enteros con signo.

Ejemplos

A continuación se presentan ejemplos que trabajan con números con signo representados con 8 bits:

- Supongamos que se realiza la suma $64 + 64$:

$$\begin{array}{r}
 \\
 \\
 + \\
 \hline
 1
 \end{array}$$

✗

El resultado es incorrecto si se interpretan los operandos como números *signed*, ya que $64 + 64 \neq -128$; es decir, se sumaron dos números positivos y el resultado fue negativo. Sin embargo, el resultado es correcto si se interpretan como números *unsigned*, dado que $64 + 64 = 128$. En este ejemplo, las banderas quedan en **CF=0** y **OF=1**.

- Supongamos que se realiza la suma $(-128) + (-128)$:

$$\begin{array}{r}
 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 + \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 \hline
 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad \textcolor{red}{X}
 \end{array}$$

El resultado es incorrecto, ya que $-128 + (-128) \neq 0$. El resultado también es incorrecto si se interpretan como números *unsigned*, dado que $128 + 128 \neq 0$. En este ejemplo, las banderas quedan en **CF=1** y **OF=1**.

- Supongamos que se realiza la resta $(-96) - 64$:

$$\begin{array}{r}
 1 \\
 1 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 - \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 \hline
 0 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad \textcolor{red}{X}
 \end{array}$$

El resultado es incorrecto, ya que $(-96) - 64 \neq 96$. Sin embargo, el resultado es correcto si se interpretan como números *unsigned*, dado que $160 - 64 = 96$. En este ejemplo, las banderas quedan en **CF=0** y **OF=1**. Esta resta también puede expresarse como una suma utilizando el complemento a dos del sustraendo:

$$\begin{array}{r}
 1 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 + \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\
 \hline
 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad \textcolor{red}{X}
 \end{array}$$

Notar que, en este caso, **CF=1** porque hubo un acarreo.

Conclusión

La bandera *overflow* se activa cuando el signo del resultado no tiene sentido matemático según los operandos:

- Positivo + Positivo = Negativo (¡Error!) $\rightarrow$ **OF = 1**
- Negativo + Negativo = Positivo (¡Error!) $\rightarrow$ **OF = 1**

Observación

En muchas arquitecturas (por ejemplo, x86), las operaciones aritméticas actualizan simultáneamente las banderas de estado, incluyendo *carry* (CF) y *overflow* (OF).

La interpretación de estas banderas depende de cómo se consideren los operandos. Para enteros sin signo, la bandera *carry* indica desbordamiento, mientras que para enteros con signo, la bandera *overflow* indica si el resultado excede el rango representable.

10. Operadores a nivel de bits

Los operadores de bits permiten manipular directamente los bits que conforman los datos de tipo entero. Estas operaciones son fundamentales en programación de bajo nivel, ya que proporcionan un control preciso sobre la representación binaria de los valores, lo que resulta útil en optimización de código, manipulación de registros, criptografía y control de hardware, entre otras aplicaciones.

Comprender el uso de estos operadores es clave para desarrollar programas eficientes y aprovechar al máximo los recursos del hardware. A lo largo de este material, exploraremos sus características, aplicaciones y ejemplos prácticos que ilustran su utilidad en distintos contextos.

Operadores a nivel de bits en C

El lenguaje C no define un tipo de dato específico para trabajar con bits, pero permite operar a nivel de bits utilizando variables de tipo entero. Para ello, ofrece un conjunto de operadores bit a bit que permiten realizar desplazamientos, máscaras y combinaciones lógicas entre bits.

La siguiente tabla presenta los principales operadores de bits (*bitwise operators*) en el lenguaje C:

Operador	Acción
&	Operación AND bit a bit
	Operación OR bit a bit
^	Operación XOR bit a bit
~	Operación NOT bit a bit
>>	Desplazamiento a la derecha
<<	Desplazamiento a la izquierda

Observación

Los operadores de bits pueden aplicarse a variables de tipo entero, tanto con signo como sin signo (`char`, `short`, `int`, `long`, etc.), y no pueden aplicarse a variables de tipo flotante (`float`, `double`, etc.). No obstante, es posible manipular la representación binaria de un número flotante reinterpretando su contenido como un entero del mismo tamaño (por ejemplo, mediante punteros, `union` o `memcpy`), lo cual veremos en el próximo apunte.

10.1. Operador AND

El operador AND bit a bit está definido por la siguiente tabla de verdad:

a_i	b_i	$a_i \& b_i$
0	0	0
0	1	0
1	0	0
1	1	1

Supongamos que tenemos dos variables, $A=56$ y $B=72$, entonces la operación AND entre estas dos variables resulta:

$$\begin{array}{r} \\ \& \\ \hline \end{array}$$

Es decir, $A\&B=8$.

Notar que el resultado hubiera sido diferente si se hubiera aplicado el operador lógico `&&`. En este caso habría resultado $A\&\&B=1$. El operador AND lógico (`&&`) devuelve el valor booleano *true* si ambos operandos son *true*. En caso contrario, devuelve *false*. Los operandos se convierten implícitamente al tipo *bool* antes de su evaluación y el resultado de la operación es de tipo *bool*.

Una aplicación útil para el operador `&` consiste en determinar si un determinado bit de cierto número es 1 o 0.

Ejemplo

Si se tiene el número $A=72$ y se quiere averiguar si el cuarto bit de dicho número es 1 o 0, podemos aplicar el operador $\&$ realizando la operación $A\&A$, donde B es un número cuya representación binaria es $00001000b$, es decir un número cuyo cuarto bit es 1 y todos los demás son ceros. A este último número se lo denomina “máscara”. Las “máscaras” son secuencias de bits que combinadas en una operación binaria con cualquier otra secuencia de bits permite modificar esta última u obtener alguna información sobre ella. Entonces:

$$\begin{array}{rcccccccc} & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & A=72 \\ \& & 0 & 0 & 0 & 0 & 1 & 0 & 0 & \text{Máscara} \\ \hline & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{array}$$

Por lo tanto, al realizar $A\&B$ vemos que efectivamente el cuarto bit del número es 1. Ya veremos que este ejemplo se puede mejorar utilizando el operador desplazamiento.

Ejemplo

Un número es potencia de 2 si tiene exactamente un bit en 1, lo que se puede verificar con $n \& (n - 1)$:

```
#include <stdio.h>

int esPotenciaDeDos(int num) {
    return (num > 0) && ((num & (num - 1)) == 0);
}

int main() {
    int num = 16;

    if (esPotenciaDeDos(num))
        printf("%d es una potencia de 2\n", num);
    else
        printf("%d no es una potencia de 2\n", num);
    return 0;
}
```

10.2. Operador OR

El operador OR bit a bit está definido por la siguiente tabla de verdad:

a_i	b_i	$a_i \mid b_i$
0	0	0
0	1	1
1	0	1
1	1	1

Supongamos que tenemos dos variables, $A=56$ y $B=72$. Entonces la operación OR entre estas dos variables resulta:

$$\begin{array}{rcccccccc} & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & (56) \\ \mid & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & (72) \\ \hline & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & (120) \end{array}$$

Es decir, $A \mid B = 120$. Notar que el resultado es distinto al obtenido si se utiliza el operador OR lógico ($\mid\mid$).

El operador $|$ se puede utilizar para “encender” determinados bits de un número.

Ejemplo

Sea el número $A=28$, si queremos poner en uno el séptimo bit podemos realizar $A=A|B$, donde $b=01000000b$:

$$\begin{array}{rcccccccc}
 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & A=28 \\
 | & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & \text{Máscara} \\
 \hline
 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 &
 \end{array}$$

Podemos observar que el séptimo bit de A ahora es uno.

10.3. Operador XOR

El operador XOR bit a bit está definida por la siguiente tabla de verdad:

a_i	b_i	$a_i \wedge b_i$
0	0	0
0	1	1
1	0	1
1	1	0

Supongamos que tenemos dos variables, $A=56$ y $B=72$, entonces la operación XOR entre estas dos variables resulta:

$$\begin{array}{rcccccccc}
 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & (56) \\
 \wedge & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & (72) \\
 \hline
 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & (112)
 \end{array}$$

Es decir, $A \wedge B=112$.

El operador XOR se puede utilizar para invertir determinados bits de un número.

Ejemplo

Sea el número $A=28$, si queremos invertir el cuarto bit podemos realizar $A=A \wedge B$, donde $B=00001000b$:

$$\begin{array}{rcccccccc} & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ \wedge & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ \hline & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \end{array}$$

Vemos que el cuarto bit de A pasó de ser uno a ser cero.

10.4. Operador NOT

El operador binario $\sim$ se conoce como como operador NOT o también operador complemento a uno. Es un operador unario, es decir solo necesita un operando. La tabla de la verdad de este operador es la siguiente:

a_i	$\sim a_i$
0	1
1	0

Por lo tanto, si tenemos la variable $A=56$, resulta $\sim A=-57$ dado que la secuencia de bits que representa a la variable A es $00111000b$ y entonces la secuencia que resulta de aplicarle el operador $\sim$ resulta $11000111b$.

Ejemplo

La expresión $x=x\&\sim 0xff$ pone los últimos 8 bits de x en cero. Notar que esto es independiente de la longitud del tipo de dato a diferencia de la expresión $x=x\&0xffffffff00$ que asume que x tiene 32 bits.

10.5. Propiedades del operador XOR

A partir de la tabla de la verdad del operador XOR se pueden determinar las siguientes propiedades:

- Conmutativa: $A \wedge B = B \wedge A$

- Asociativa: $A \wedge (B \wedge C) = (A \wedge B) \wedge C$
- $A \wedge A = 0$
- $A \wedge 0 = A$

A partir de las propiedades anteriores resulta:

- $(B \wedge A) \wedge A = B \wedge 0 = B$

Esto resulta útil para hacer cifrado de datos. Una cadena de texto puede ser cifrada aplicando el operador de bit XOR sobre cada uno de los caracteres utilizando una clave. Para descifrar la salida, solo hay que volver a aplicar el operador XOR con la misma clave.

Ejemplo

La cadena de caracteres “Hola” se puede representar utilizando código ASCII como 01001000 01101111 01101100 01100001. Esta cadena puede ser cifrada con la clave 11110011 de la siguiente manera:

$$\begin{array}{r} 01001000 \ 01101111 \ 01101100 \ 01100001 \\ \sim \ 11110011 \ 11110011 \ 11110011 \ 11110011 \\ \hline 10111011 \ 10011100 \ 10011111 \ 10010010 \end{array}$$

Si a este resultado se le vuelve a aplicar el operador XOR con la misma clave volvemos a tener la cadena original:

$$\begin{array}{r} 10111011 \ 10011100 \ 10011111 \ 10010010 \\ \sim \ 11110011 \ 11110011 \ 11110011 \ 11110011 \\ \hline 01001000 \ 01101111 \ 01101100 \ 01100001 \end{array}$$

10.6. Desplazamiento de bits

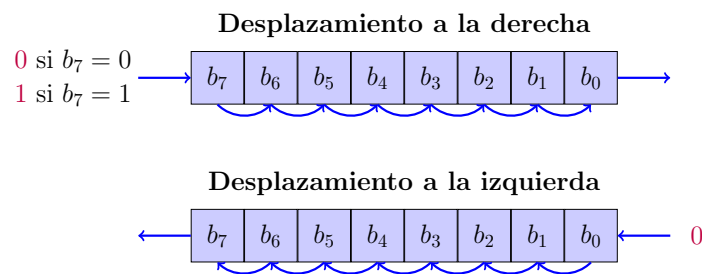
Se pueden realizar desplazamientos de N bits, con $N \in \mathbb{N}$, es decir $N = 1, 2, \dots$. Los desplazamientos de bits pueden ser en dos direcciones: desplazamiento hacia la derecha o desplazamiento hacia la izquierda. Cada vez que se hace un desplazamiento se completa con ceros o unos en el otro lado (no se trata de una rotación).

- En C, al realizar un desplazamiento a la izquierda ($\ll$), siempre ingresan ceros por la derecha, independientemente de si la variable es con signo o sin signo.
- Desplazamiento a la derecha ($\gg$):
 - Si el operando es *unsigned*, entran ceros (desplazamiento lógico).
 - Si el operando es *signed*, el estándar C no obliga a cómo hacerlo: depende de la implementación. En la práctica, casi todos los compiladores modernos (como GCC) realizan un desplazamiento aritmético, lo que implica copiar el bit de signo: entra 1 si el número es negativo y 0 si es positivo.

La siguiente figura muestra los desplazamientos de un bit en un valor *unsigned* de 8 bits:



La siguiente figura muestra los desplazamientos de un bit en un valor *signed* de 8 bits:



Supongamos que tenemos el número $A = 56$ ($(56)_{10} = 0b00111000$) y le aplicamos un corrimiento de un bit hacia la derecha: $A \gg 1$. El resultado es $0b00011100 = (28)_{10}$. Ahora supongamos que $A = -128$ ($(-128)_{10} = 0b10000000$) y también le aplicamos un corrimiento de un bit hacia la derecha. El resultado es $0b11000000 = (-64)_{10}$. Es decir, se ha dividido al número a

por dos. Por lo tanto, desplazando el número n bits hacia la derecha se realiza la división entera del número:

$$A \gg n = \frac{A}{2^n}$$

Análogamente, desde el punto de vista matemático, el desplazamiento a la izquierda equivale a multiplicar por potencias de dos:

$$A \ll n = A \times 2^n \quad (3)$$

La ventaja de los desplazamientos es que son menos costosos que hacer las operaciones de multiplicación y división.

Importante:

- Hay que tener cuidado al realizar desplazamientos hacia la izquierda dado que se obtienen resultados erróneos (con respecto a la Ecuación (3)) debido al límite en el rango de valores representables. Es decir, desde el punto de vista de Arquitectura del Computador, el desplazamiento a la izquierda NO es equivalente a multiplicar por potencias de dos.
- También es necesario tener precaución al realizar desplazamientos hacia la derecha, ya que se pierde la parte fraccionaria del resultado.
- Existen dos tipos de desplazamientos hacia la derecha:
 - El “*arithmetic shift*” se completa de acuerdo al bit de signo.
 - El “*logical shift*” se completa con ceros.

Ejemplo

En el Ejemplo 10.1 vimos cómo determinar si un bit determinado de un número es 0 o 1. Dicho ejemplo se puede mejorar realizando un desplazamiento hacia la derecha luego de aplicar el operador $\&$:

$(A \& B) \gg 3$

El resultado es 1, indicando que el cuarto bit es efectivamente 1.

Ejemplo

El siguiente código cuenta cuántos bits están en 1 en un número:

```
#include <stdio.h>

int contarBits(int num) {
    int count = 0;
    while (num) {
        count += num & 1; // Sumar si el bit menos
                           // significativo es 1
        num >>= 1;        // Desplazar a la derecha
    }
    return count;
}

int main() {
    int num = 10;
    printf("El número %d tiene %d bits en 1\n", num,
        contarBits(num));
    return 0;
}
```

Ejemplo

El desplazamiento de bits es una alternativa eficiente para multiplicar o dividir por potencias de 2:

```
#include <stdio.h>

int main() {
    int num = 6;
    printf("Multiplicar por 2: %d<<1 = %d\n", num, num
        <<1);
    printf("Dividir por 2: %d>>1 = %d\n", num, num>>1);
    return 0;
}
```

Sin embargo, hay que tener algunas precauciones:

- Asegurar que el resultado no supere el tamaño del tipo de dato.
- Verificar si el compilador usa desplazamiento aritmético o lógico para números negativos.
- Tener cuidado con el truncamiento en la división cuando el número no es par.
- No desplazar más bits de los que tiene el tipo de dato.

A. Apéndices

A.1. Representación de caracteres

Además de los datos numéricos, también es necesario almacenar datos simbólicos. Estos datos, también llamados no numéricos, permiten representar mensajes como “Hola mundo”. Si bien dichos símbolos son fácilmente comprensibles para los hablantes de un determinado idioma, la memoria de la computadora está diseñada para almacenar y procesar números. Por lo tanto, cada símbolo se representa mediante una codificación que le asigna un valor numérico. En particular, aunque los caracteres no son datos de tipo entero, en la práctica se almacenan utilizando códigos equivalentes a enteros de un byte.

En una computadora, un caracter es una unidad de información que corresponde a un símbolo tal como una letra en el alfabeto. Los caracteres incluyen letras, dígitos numéricos, signos de puntuación comunes (como “.” o “!”) y espacios en blanco. El concepto general también incluye caracteres de control, que no corresponden a símbolos en un idioma en particular, sino a otra información utilizada para procesar texto. Ejemplos de caracteres de control incluyen el retorno de carro o la tabulación.

Los caracteres se pueden representar mediante el Código ASCII (acrónimo inglés de *American Standard Code for Information Interchange* — Código Estándar estadounidense para el Intercambio de Información). Según la tabla ASCII⁴, a cada carácter y carácter de control se le asigna un valor numérico. Cuando se utiliza ASCII, el carácter que se muestra se basa en el valor numérico asignado. Esto solo funciona si todos están de acuerdo con los valores comunes, que es el propósito de la tabla ASCII. Por ejemplo, la letra “A” se define como $(65)_{10} = 0x41$. Este número se almacena en la memoria de la computadora y, cuando se muestra en la consola, se muestra la letra “A”. Existen otros estándares ampliamente utilizados, por ejemplo el estándar UNICODE⁵.

Los símbolos numéricos también se pueden representar en ASCII. Por ejemplo, “9” se representa como $(57)_{10} = 0x39$ en la memoria de la computadora. El “9” se puede mostrar como salida a la consola. Si se envía a la consola, el valor entero $(9)_{10}$ se interpretaría como un valor ASCII que, en este caso, sería una tabulación. Es muy importante comprender la diferencia entre caracteres (como “2”) y números enteros (como $(2)_{10}$). Los caracteres se pueden mostrar en la consola, pero no se pueden usar para cálculos. Los números enteros se pueden utilizar para los cálculos, pero no se pueden mostrar en la consola (sin cambiar la representación).

Normalmente, un carácter se almacena en un byte (8 bits) de espacio. Esto funciona bien ya que la memoria es direccionable por bytes. Con 8 bits es posible representar hasta $2^8 = 256$ valores distintos, lo que permite codificar un conjunto de hasta 256 caracteres diferentes.

A.2. Representación de cadenas

Una cadena es una serie de caracteres ASCII, normalmente terminados en NULL. El NULL es un carácter de control ASCII no imprimible. Dado que no es imprimible, se puede utilizar para marcar el final de una cadena. Por ejemplo, la cadena “Hello” se representaría de la siguiente manera:

⁴Consulte <https://www.asciitable.com/> para ver una tabla ASCII completa.

⁵Para mayor información, consultar: <http://en.wikipedia.org/wiki/Unicode>.

Caracter	“H”	“e”	“I”	“l”	“o”	NULL
Valor ASCII (decimal)	72	101	108	108	111	0
Valor ASCII (hexadecimal)	0x48	0x65	0x6C	0x6C	0x6F	0x0

Una cadena puede consistir parcial o completamente en símbolos numéricos. Por ejemplo, la cadena “19653” se representaría de la siguiente manera:

Caracter	“1”	“9”	“6”	“5”	“3”	NULL
Valor ASCII (decimal)	49	57	54	53	51	0
Valor ASCII (hexadecimal)	0x31	0x39	0x36	0x35	0x33	0x0

Nuevamente, es muy importante comprender la diferencia entre la cadena “19653” (que necesita 6 bytes para ser almacenada) y el entero 19653_{10} (que se puede almacenar con 2 bytes).

A.3. Tamaños de tipos numéricos en arquitecturas x86-64

Tipo	Tamaño (bytes)	Bits
char	1	8
short	2	16
int	4	32
long	8	64
long long	8	64
float	4	32
double	8	64
long double	16	80

Nota: En muchas arquitecturas x86-64, **long double** utiliza 80 bits de precisión extendida (formato IEEE 754), aunque se almacena en 16 bytes.

Observación

El estándar del lenguaje C no fija el tamaño exacto de los tipos numéricos básicos (**char**, **int**, **long**, etc.), sino que establece únicamente relaciones entre ellos (por ejemplo, `sizeof(short) ≤ sizeof(int) ≤ sizeof(long)`). En consecuencia, el tamaño concreto de estos tipos depende de la arquitectura y del modelo de datos utilizado, aunque en la práctica existen convenciones ampliamente adoptadas.

Esta característica tiene implicancias importantes:

- En programación de bajo nivel (por ejemplo, en arquitectura de computadoras o sistemas), es recomendable utilizar los tipos de tamaño fijo definidos en `<stdint.h>` (como `int32_t` o `uint64_t`), ya que permiten especificar con precisión la cantidad de bits utilizada.
- En programación general, el uso de tipos básicos (`int`, `long`, etc.) suele ser suficiente y permite que el compilador elija la representación más eficiente para la plataforma.

Referencias

- [1] Andrew S. Tanenbaum , *Organización de computadoras: Un enfoque estructurado*, cuarta edición, Pearson Education, 2000
- [2] Paul A. Carter, *PC Assembly Language*, Disponible en formato electrónico: <http://www.drpaulcarter.com/pcasm/>, 2006.
- [3] M. Morris Mano, *Computer system architecture*, tercera edición, Prentice-Hall, 1993.
- [4] Randall Hyde, *The art of assembly language*, segunda edición, No Starch Pr, 2003.
- [5] Ed Jorgensen, *x86-64 Assembly Language Programming with Ubuntu*, 2020.